

Tuyển tập Bài tập: Hàng đợi Ưu tiên & Cấu trúc Đống (Heap)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 **Duc-Minh Vu**. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Bảng Tổng hợp Bài tập Tuyển chọn

STT	Bài toán	Nền tảng	Độ khó	Kỹ thuật trọng tâm
1	Kth Largest in a Stream	LeetCode 703	★ Cơ bản	Min-Heap kích thước K duy trì phần tử lớn thứ K
2	Last Stone Weight	LeetCode 1046	★ Cơ bản	Max-Heap mô phỏng va chạm hai hòn đá lớn nhất
3	Room Allocation	CSES 1164	★ Cơ bản	Sweep-line + Min-Heap quản lý phòng trống trả sớm
4	Kth Largest Element in an Array	LeetCode 215	★ Cơ bản	Min-Heap size K thời gian $O(N \log K)$
5	Top K Frequent Elements	LeetCode 347	★ Cơ bản	Đếm tần suất + Min-Heap lọc K phần tử phổ biến nhất
6	K Closest Points to Origin	LeetCode 973	★ Cơ bản	Max-Heap duy trì K điểm có khoảng cách Euclidean nhỏ nhất
7	Shortest Routes I	CSES 1671	★★ Trung cấp	Thuật toán Dijkstra tìm đường đi ngắn nhất $O(M \log N)$
8	Find Median from Data Stream	LeetCode 295	★★ Trung cấp	Kỹ thuật Hai Heaps đối đỉnh duy trì trung vị động
9	Stick Divisions	CSES 1161	★★ Trung cấp	Tham lam Huffman / Nối que gỗ bằng Min-Heap $O(N \log N)$
10	Find K Pairs with Smallest Sums	LeetCode 373	★★ Trung cấp	Min-Heap mở rộng đa hướng $O(K \log K)$

STT	Bài toán	Nền tảng	Độ khó	Kỹ thuật trọng tâm
11	Flight Discount	CSES 1195	★★ Trung cấp	Dijkstra 2 trạng thái (đã dùng / chưa dùng vé giảm giá)
12	Task Scheduler	LeetCode 621	★★ Trung cấp	Max-Heap tham lam tần suất cao nhất + Hàng đợi hồi chiêu
13	Merge k Sorted Lists	LeetCode 23	★★★ Nâng cao	Hợp nhất K danh sách liên kết bằng Min-Heap $O(N \log K)$
14	Flight Routes	CSES 1196	★★★ Nâng cao	K Shortest Paths: Mở rộng Dijkstra duy trì K khoảng cách
15	Trapping Rain Water II	LeetCode 407	★★★ Nâng cao	Min-Heap 2D + Thuật toán loang biên mặt nước $O(MN \log MN)$
16	Playlist	Codeforces 1140C	★★★ Nâng cao	Sắp xếp theo vẻ đẹp + Min-Heap duy trì top K độ dài



HƯỚNG DẪN GIẢI CHI TIẾT & GỢI Ý CHIẾN THUẬT

1. Nhóm Bài tập Cơ bản (6 bài)

Bài 1: [LeetCode 703 - Kth Largest Element in a Stream](#)

- **Đề bài tóm tắt:** Thiết kế lớp nhận luồng số nguyên và liên tục trả về phần tử lớn thứ K hiện có.
- **Phân tích & Thuật toán:**
- Duy trì một **Min-Heap kích thước tối đa K** : `priority_queue<int, vector<int>, greater<int>> min_pq;`
- Mỗi khi thêm số mới x :
 - Nếu kích thước $< K$, đẩy x vào heap.
 - Nếu kích thước $= K$ và $x > \text{min_pq.top}()$, `pop()` phần tử nhỏ nhất ra và `push(x)`.
- Phần tử nằm tại `min_pq.top()` chính là phần tử lớn thứ K . Thời gian: $O(\log K)$ mỗi thao tác.

Bài 2: [LeetCode 1046 - Last Stone Weight](#)

- **Đề bài tóm tắt:** Cho mảng trọng lượng các hòn đá. Mỗi lượt lấy 2 hòn đá nặng nhất đập vào nhau: nếu bằng nhau thì cả hai vỡ vụn, nếu khác nhau thì hòn đá mới có trọng lượng bằng hiệu số còn lại. Tìm trọng lượng hòn đá cuối cùng (hoặc 0 nếu hết).
- **Phân tích & Thuật toán:**
- Đưa toàn bộ vào Max-Heap `priority_queue<int> pq;`
- Mỗi bước rút 2 phần tử lớn nhất: `y = pq.top(); pq.pop(); x = pq.top(); pq.pop();`
- Nếu $y > x$, đẩy $y - x$ trở lại Heap.

- Lặp lại cho đến khi `pq.size() <= 1`.
-

Bài 3: [CSES 1164 - Room Allocation](#)

- **Đề bài tóm tắt:** Phân bổ phòng khách sạn cho N khách hàng sao cho số phòng cần dùng là ít nhất.
 - **Phân tích & Thuật toán:**
 - Sắp xếp các khách hàng theo thời điểm đến.
 - Dùng **Min-Heap** lưu các phòng đang có người ở theo cặp (thời điểm trả phòng, số phòng).
 - Khi khách mới đến tại ngày A_i :
 - Nếu `heap.top().first < A_i`: phòng đã trống, lấy lại số phòng đó và cập nhật thời điểm trả mới.
 - Ngược lại: cấp một phòng hoàn toàn mới `++roomCount`.
 - Độ phức tạp: $O(N \log N)$.
-

Bài 4: [LeetCode 215 - Kth Largest Element in an Array](#)

- **Đề bài tóm tắt:** Tìm phần tử lớn thứ K trong mảng số nguyên chưa được sắp xếp.
 - **Phân tích & Thuật toán:**
 - Dùng Min-Heap duy trì K phần tử lớn nhất đã duyệt:
 - Duyệt qua N phần tử: đẩy vào `min_pq`. Nếu `min_pq.size() > K`, gọi `min_pq.pop()`.
 - Kết thúc, `min_pq.top()` là đáp án.
 - Độ phức tạp: $O(N \log K)$ thời gian, $O(K)$ bộ nhớ phụ trợ.
-

Bài 5: [LeetCode 347 - Top K Frequent Elements](#)

- **Đề bài tóm tắt:** Cho mảng số nguyên, tìm K phần tử có tần suất xuất hiện nhiều nhất.
 - **Phân tích & Thuật toán:**
 - Dùng Hash Map đếm tần suất của từng số.
 - Dùng Min-Heap kích thước K lưu cặp (tần suất, giá trị).
 - Duyệt qua các cặp trong map, duy trì K phần tử có tần suất lớn nhất.
 - Độ phức tạp: $O(N \log K)$.
-

Bài 6: [LeetCode 973 - K Closest Points to Origin](#)

- **Đề bài tóm tắt:** Cho danh sách các điểm trên mặt phẳng Oxy , tìm K điểm có khoảng cách Euclidean tới gốc tọa độ $(0, 0)$ nhỏ nhất.
 - **Phân tích & Thuật toán:**
 - Khoảng cách bình phương: $d = x^2 + y^2$.
 - Dùng **Max-Heap** kích thước K : lưu trữ K điểm gần nhất. Khi gặp một điểm mới gần hơn điểm xa nhất trong heap (`d < max_pq.top().dist`), thay thế đỉnh heap.
 - Độ phức tạp: $O(N \log K)$.
-

2. Nhóm Bài tập Trung cấp (6 bài)

Bài 7: [CSES 1671 - Shortest Routes I](#)

- **Đề bài tóm tắt:** Tìm đường đi ngắn nhất từ thành phố 1 đến mọi thành phố khác trên đồ thị có hướng N đỉnh và M chuyến bay với chi phí không âm ($N, M \leq 2 \cdot 10^5$).
 - **Phân tích & Thuật toán:**
 - Thuật toán **Dijkstra chuẩn với Min-Heap**:
 - `priority_queue<pair<long long, int>, vector<pair<long long, int>>, greater<pair<long long, int>>> pq;`
 - Mảng `dist` khởi tạo bằng ∞ , `dist[1] = 0`.
 - Kiểm tra bỏ qua đỉnh lỗi thời: `if (d > dist[u]) continue;`
 - Độ phức tạp: $O(M \log N)$. Lưu ý dùng `long long` tránh tràn số.
-

Bài 8: [LeetCode 295 - Find Median from Data Stream](#)

- **Đề bài tóm tắt:** Thiết kế cấu trúc dữ liệu nhận luồng số và hỗ trợ truy vấn trung vị trong $O(1)$.
 - **Phân tích & Thuật toán:**
 - Dùng **Hai Heaps đối đỉnh**:
 - `max_heap`: Lưu nửa nhỏ các số.
 - `min_heap`: Lưu nửa lớn các số.
 - Luôn duy trì: `size(max_heap) = size(min_heap)` hoặc `size(max_heap) = size(min_heap) + 1`.
 - Truy vấn trung vị: Lấy trực tiếp từ đỉnh heap trong $O(1)$.
-

Bài 9: [CSES 1161 - Stick Divisions](#)

- **Đề bài tóm tắt:** Bạn có một thanh gỗ độ dài X cần chia thành N thanh có độ dài $d_1, \dots, d_n$. Chi phí mỗi lần chia một thanh gỗ bằng độ dài của thanh đó. Tìm tổng chi phí tối thiểu.
 - **Phân tích & Thuật toán:**
 - Đảo ngược bài toán: Bắt đầu từ N thanh gỗ nhỏ, tìm chi phí tối thiểu để ghép chúng lại thành 1 thanh lớn (Bài toán Mã hóa Huffman).
 - Đưa tất cả độ dài vào **Min-Heap**:
 - Mỗi bước lấy ra 2 thanh ngắn nhất A và B .
 - Chi phí ghép là $A + B$, đẩy $A + B$ trở lại Min-Heap.
 - Lặp lại $N - 1$ lần. Tổng chi phí là kết quả cần tìm.
 - Độ phức tạp: $O(N \log N)$.
-

Bài 10: [LeetCode 373 - Find K Pairs with Smallest Sums](#)

- **Đề bài tóm tắt:** Cho hai mảng đã sắp xếp tăng dần A và B . Tìm K cặp $(A[i], B[j])$ có tổng nhỏ nhất.
- **Phân tích & Thuật toán:**
- Đưa các cặp ban đầu $(A[i], B[0])$ với $0 \leq i < \min(N, K)$ vào Min-Heap theo tổng $A[i] + B[0]$.

- Mỗi bước rút cặp nhỏ nhất $(A[i], B[j])$ ra, ghi nhận kết quả.
- Đẩy cặp tiếp theo $(A[i], B[j + 1])$ vào heap nếu $j + 1 < M$.
- Độ phức tạp: $O(K \log K)$.

Bài 11: [CSES 1195 - Flight Discount](#)

- **Đề bài tóm tắt:** Bạn muốn bay từ thành phố 1 đến thành phố N . Bạn có một phiếu giảm giá 50% giá vé cho đúng một chuyến bay bất kỳ ($\lfloor c/2 \rfloor$). Tìm chi phí bay nhỏ nhất.
- **Phân tích & Thuật toán:**
- Xây dựng đồ thị 2 tầng (Dijkstra đa trạng thái):
 - Trạng thái $(u, 0)$: Đang ở đỉnh u và chưa sử dụng phiếu giảm giá.
 - Trạng thái $(u, 1)$: Đang ở đỉnh u và đã sử dụng phiếu giảm giá.
- Các bước chuyển:
 - Từ $(u, 0)$ sang $(v, 0)$ với trọng số w (không dùng vé).
 - Từ $(u, 0)$ sang $(v, 1)$ với trọng số $\lfloor w/2 \rfloor$ (dùng vé cho chuyến bay này).
 - Từ $(u, 1)$ sang $(v, 1)$ với trọng số w (sau khi đã dùng vé).
- Chạy Dijkstra trên đồ thị $2N$ đỉnh và $3M$ cạnh. Kết quả: $\text{dist}[N][1]$.

Bài 12: [LeetCode 621 - Task Scheduler](#)

- **Đề bài tóm tắt:** Có các tác vụ cần thực thi trên CPU. Hai tác vụ cùng loại phải cách nhau ít nhất khoảng thời gian N . Tìm thời gian tối thiểu để hoàn thành tất cả tác vụ.
- **Phân tích & Thuật toán:**
- Tham lam: Luôn ưu tiên thực hiện tác vụ có tần suất xuất hiện nhiều nhất còn lại bằng Max-Heap.
- Dùng Queue lưu các tác vụ đang trong thời gian hồi chiêu (tần suất còn lại, thời điểm có thể thực thi lại).

3. Nhóm Bài tập Nâng cao (4 bài)

Bài 13: [LeetCode 23 - Merge k Sorted Lists](#)

- **Đề bài tóm tắt:** Hợp nhất K danh sách liên kết đã sắp xếp tăng dần thành một danh sách sắp xếp duy nhất.
- **Phân tích & Thuật toán:**
- Dùng Min-Heap lưu con trỏ đầu tiên của mỗi danh sách không rỗng: `priority_queue` theo `node->val`.
- Mỗi bước rút nút nhỏ nhất ra, nối vào đuôi danh sách kết quả, đẩy `node->next` vào heap nếu còn.
- Tổng số phần tử là N . Độ phức tạp: $O(N \log K)$.

Bài 14: [CSES 1196 - Flight Routes](#)

- **Đề bài tóm tắt:** Tìm độ dài của K đường đi ngắn nhất từ thành phố 1 đến thành phố N trên đồ thị có hướng ($K \leq 10, N, M \leq 10^5$).
 - **Phân tích & Thuật toán:**
 - Mở rộng thuật toán Dijkstra:
 - Thay vì chỉ duy trì 1 khoảng cách ngắn nhất cho mỗi đỉnh, ta duy trì mảng `count[u]` đếm số lần đỉnh u được rút ra khỏi Min-Heap.
 - Một đỉnh u có thể được mở rộng tối đa K lần (mỗi lần là đường đi ngắn thứ $1, 2, \dots, K$ tới u).
 - Nếu `count[u] > K`, bỏ qua đỉnh đó.
 - Khi đỉnh N được rút ra lần thứ K , ta có đủ K đường đi ngắn nhất.
 - Độ phức tạp: $O(K \cdot M \log(KM))$.
-

Bài 15: [LeetCode 407 - Trapping Rain Water II](#)

- **Đề bài tóm tắt:** Cho bản đồ độ cao $N \times M$ dạng lưới 2D. Tính tổng lượng nước mưa tối đa có thể giữ lại được.
 - **Phân tích & Thuật toán:**
 - Khái niệm "Mặt nước tràn từ biên vào":
 - Đẩy toàn bộ các ô nằm trên đường biên của lưới vào **Min-Heap** lưu `{chiều cao, x, y}`.
 - Mỗi bước rút ô có độ cao thấp nhất ra (đây là điểm tràn thấp nhất hiện tại).
 - Duyệt 4 ô kề láng giềng: nếu láng giềng thấp hơn mặt nước hiện tại, lượng nước đọng bằng `nước_tràn - chiều_cao_ô`. Đẩy láng giềng vào heap với độ cao mới bằng `max(độ cao ô, nước tràn)`.
 - Độ phức tạp: $O(NM \log(NM))$.
-

Bài 16: [Codeforces 1140C - Playlist](#)

- **Đề bài tóm tắt:** Có N bài hát, bài thứ i có độ dài t_i và độ hay b_i . Chọn tối đa K bài hát để tối đa hóa:

$$\text{Độ sướng} = \left(\sum_{i \in S} t_i \right) \times \min_{i \in S} b_i$$

- **Phân tích & Thuật toán:**
 - Sắp xếp các bài hát theo độ hay b_i giảm dần.
 - Khi duyệt qua bài hát i , ta cố định b_i là độ hay nhỏ nhất trong tập hợp chọn.
 - Để tối đa hóa tổng độ dài của tối đa K bài hát trong số các bài có độ hay $\geq b_i$: Dùng **Min-Heap** lưu độ dài t của tối đa K bài hát:
 - Đẩy t_i vào Min-Heap, cộng vào tổng `sum_t`.
 - Nếu `heap.size() > K`, trừ `heap.top()` khỏi `sum_t` và `pop()`.
 - Cập nhật đáp án: `ans = max(ans, sum_t * b[i])`.
 - Độ phức tạp: $O(N \log N)$.
-



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.